

Федеральное государственное автономное образовательное учреждение высшего  
образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и Компьютерной техники

Лабораторная работа №2  
Вариант 31077

Выполнила:

Косогова Мария Александровна

Группа: Р3106

Проверил:

Вербовой Александр

Александрович,

Преподаватель практики.

Санкт-Петербург 2024

## **Содержание**

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>ЗАДАНИЯ .....</b>                                         | <b>3</b>  |
| <b>ДИАГРАММА КЛАССОВ РЕАЛИЗОВАННОЙ ОБЪЕКТНОЙ МОДЕЛИ.....</b> | <b>4</b>  |
| <b>ОСНОВНЫЕ ЭТАПЫ РАБОТЫ.....</b>                            | <b>5</b>  |
| <b>ВЫВОД .....</b>                                           | <b>16</b> |

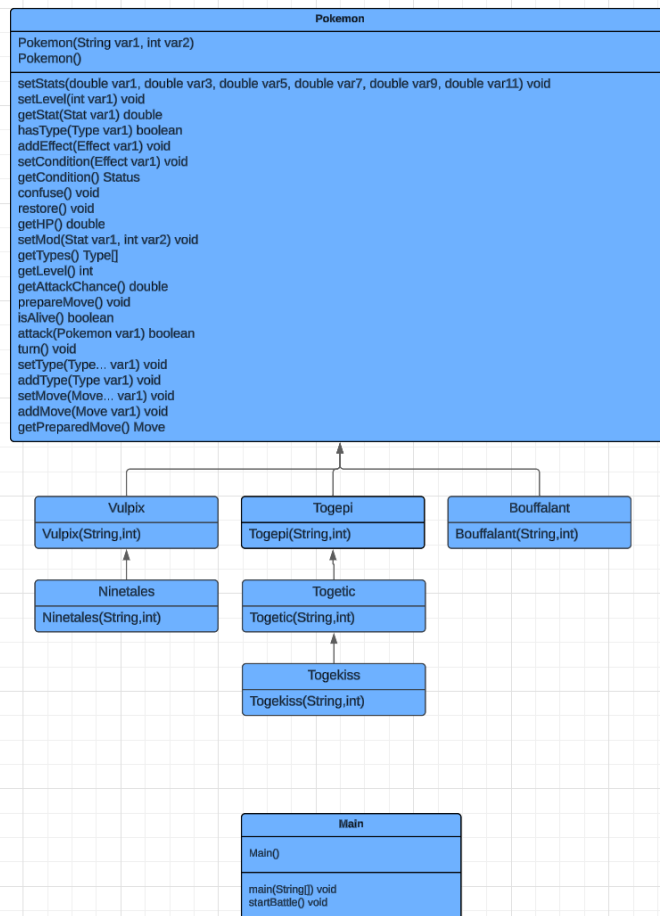
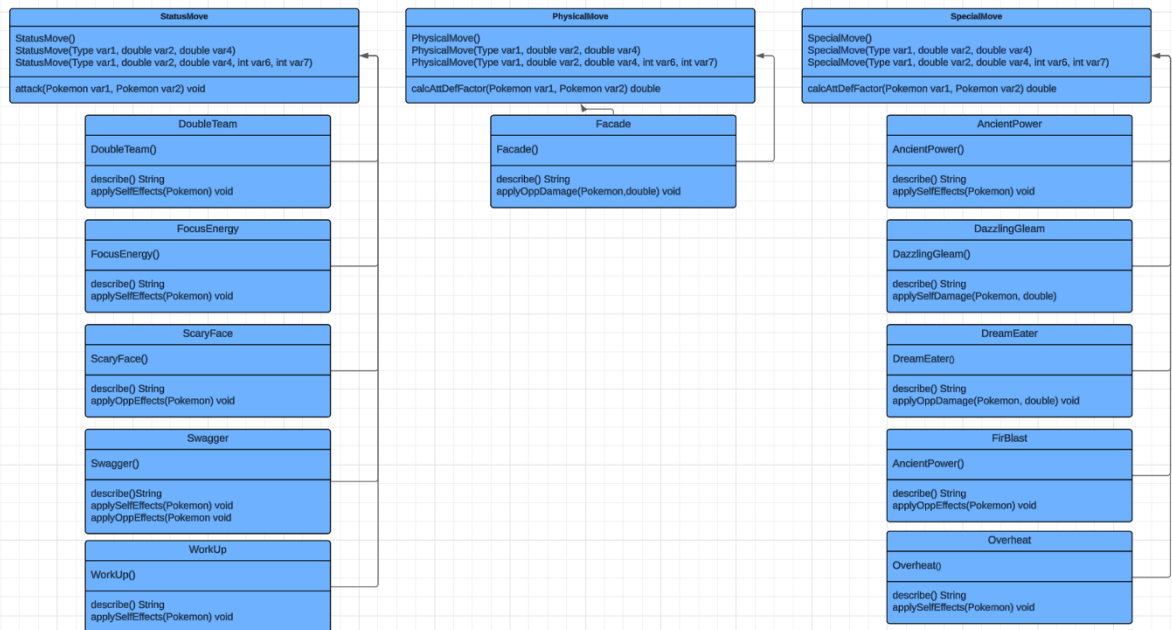
## Задания

1. Ознакомиться с документацией, обращая особое внимание на классы `Pokemon` и `Move`. При дальнейшем выполнении лабораторной работы читать документацию еще несколько раз.
2. Скачать файл `Pokemon.jar`. Его необходимо будет использовать как для компиляции, так и для запуска программы. Распаковывать его не надо! Нужно научиться подключать внешние `jar`-файлы к своей программе. Написать минимально работающую программу и посмотреть как она работает.  

```
Battle b = new Battle();  
Pokemon p1 = new Pokemon("Чужой", 1);  
Pokemon p2 = new Pokemon("Хищник", 1);  
b.addAlly(p1);  
b.addFoe(p2);  
b.go();
```
3. Создать один из классов покемонов для своего варианта. Класс должен наследоваться от базового класса `Pokemon`. В конструкторе нужно будет задать типы покемона и его базовые характеристики. После этого попробуйте добавить покемона в сражение.
4. Создать один из классов атак для своего варианта (лучше всего начать с физической или специальной атаки). Класс должен наследоваться от класса `PhysicalMove` или `SpecialMove`. В конструкторе нужно будет задать тип атаки, ее силу и точность. После этого добавить атаку покемону и проверить ее действие в сражении. Не забудьте переопределить метод `describe`, чтобы выводилось нужное сообщение.
5. Если действие атаки отличается от стандартного, например, покемон не промахивается, либо атакующий покемон также получает повреждение, то в классе атаки нужно дополнительно переопределить соответствующие методы (см. документацию). При реализации атак, которые меняют статус покемона (наследники `StatusMove`), скорее всего придется разобраться с классом `Effect`. Он позволяет на один или несколько ходов изменить состояние покемона или модификатор его базовых характеристик.
6. Доделать все необходимые атаки и всех покемонов, распределить покемонов по командам, запустить сражение.



## Диаграмма классов реализованной объектной модели



## Основные этапы работы

### Бой покемонов

```
import Pokemons.Bouffalant;
import Pokemons.Ninetales;
import Pokemons.Togekiss;
import Pokemons.Togepi;
import Pokemons.Togetic;
import Pokemons.Vulpix;
import ru.ifmo.se.pokemon.Battle;

public class Main {
    public Main() {
    }

    public static void main(String[] var0) { startBattle(); }

    public static void startBattle() {
        Battle var0 = new Battle();
        Bouffalant var1 = new Bouffalant( name: "", level: 1);
        Ninetales var2 = new Ninetales( name: "", level: 1);
        Togekiss var3 = new Togekiss( name: "", level: 1);
        Togepi var4 = new Togepi( name: "", level: 1);
        Togetic var5 = new Togetic( name: "", level: 1);
        Vulpix var6 = new Vulpix( name: "", level: 1);
        var0.addAlly(var1);
        var0.addAlly(var2);
        var0.addAlly(var3);
        var0.addFoe(var4);
        var0.addFoe(var5);
        var0.addFoe(var6);
        var0.go();
    }
}
```

Рисунок 1

## Покемоны

```
package Pokemons;
import Moves.*;
import ru.ifmo.se.pokemon.*;

public final class Bouffalant extends Pokemon { 1 usage  ⚙ Мария *
    public Bouffalant(String name, int level) { 1 usage  ⚙ Мария *
        super(name, level);
        setStats(v: 95, v1: 110, v2: 95, v3: 40, v4: 95, v5: 55);
        setType(Type.NORMAL);
        setMove(new Swagger(), new WorkUp(), new FocusEnergy(), new ScaryFace());
    }
}
```

Рисунок 2

```
package Pokemons;
import Moves.*;
import ru.ifmo.se.pokemon.*;
💡
public class Vulpix extends Pokemon { 2 usages  1 inheritor
    public Vulpix(String name, int level) { 2 usages
        super(name, level);
        setStats(v: 38, v1: 41, v2: 40, v3: 50, v4: 65, v5: 65);
        setType(Type.FIRE);
        setMove(new FireBlast(), new Overheat(), new Swagger());
    }
}
```

Рисунок 3

```

package Pokemons;
import Moves.*;
import ru.ifmo.se.pokemon.*;

public final class Ninetales extends Vulpix { 1 usage
    public Ninetales(String name, int level) { 1 usage
        super(name, level);
        setStats(v: 73, v1: 76, v2: 75, v3: 81, v4: 100, v5: 100);
        setType(Type.FIRE);
        setMove(new DreamEater());
    }
}

```

Рисунок 4

```

package Pokemons;
import Moves.*;
import ru.ifmo.se.pokemon.*;

public class Togepi extends Pokemon { 2 usages 2 inheritors
    public Togepi(String name, int level){ 2 usages
        super (name,level);
        setStats( v: 35, v1: 20, v2: 65, v3: 40, v4: 65, v5: 20);
        setType(Type.FAIRY);
        setMove(new DazzlingGleam(),new Facade());
    }
}

```

Рисунок 5

```

package Pokemons;
import Moves.*;
import ru.ifmo.se.pokemon.*;

public class Togetic extends Togepi { 2 usages 1 inheritor
    public Togetic(String name, int level) { 2 usages
        super(name, level);
        setStats( v: 55, v1: 40, v2: 85, v3: 80, v4: 105, v5: 40);
        setType(Type.FAIRY,Type.FLYING);
        setMove(new AncientPower());
    }
}

```

Рисунок 6

```

package Pokemons;
import Moves.*;
import ru.ifmo.se.pokemon.*;

public final class Togekiss extends Togetic { 1 usage
    public Togekiss(String name, int level) { 1 usage
        super(name, level);
        setStats(v: 85, v1: 50, v2: 95, v3: 120, v4: 115, v5: 80);
        setType(Type.FAIRY, Type.FLYING);
        setMove(new DoubleTeam());
    }
}

```

Рисунок 7

- Силы/атаки

```

package Moves;
import ru.ifmo.se.pokemon.*;

public class WorkUp extends StatusMove { 1 usage
    public WorkUp() { 1 usage
        super(Type.NORMAL, v: 0, v1: 0);
    }

    @Override
    protected String describe() {
        return "Применил Work Up";
    }

    @Override no usages
    protected void applySelfEffects(Pokemon p) {
        // Повышаем атаку на 1 стадию
        p.setMod(Stat.ATTACK, i: +1);
        // Повышаем специальную атаку на 1 стадию
        p.setMod(Stat.SPECIAL_ATTACK, i: +1);
    }
}

```

Рисунок 8



```

package Moves;
import ru.ifmo.se.pokemon.*;

public class Swagger extends StatusMove { 2 usages
    public Swagger() { 2 usages
        super(Type.NORMAL, v: 0, v1: 85);
    }

    @Override
    protected String describe() {
        return "Применил Swagger";
    }

    @Override no usages
    protected void applySelfEffects(Pokemon p) {
        // Повышаем атаку цели на 2 этапа
        p.setMod(Stat.ATTACK, i: +2);
    }

    @Override no usages
    protected void applyOppEffects(Pokemon p) {
        // Конфузим противника
        Effect.confuse(p);
    }
}

```

Рисунок 9

```

package Moves;
import ru.ifmo.se.pokemon.*;

public class ScaryFace extends StatusMove { 2 usages
    public ScaryFace(){ 2 usages
        super(Type.NORMAL, v: 0, v1: 100);
    }

    @Override
    protected String describe() {
        return "Применил Scary Face";
    }

    @Override no usages
    protected void applyOppEffects(Pokemon p) {
        // Снижение скорости цели на 2 этапа
        p.setMod(Stat.SPEED, i: -2);
    }
}

```

Рисунок 10

```

package Moves;
import ru.ifmo.se.pokemon.*;

public class Overheat extends SpecialMove { 1 usage
    public Overheat(){ 1 usage
        super(Type.FIRE, v: 130, v1: 90);
    }
    @Override
    protected String describe() {
        return "Применил Overheat";
    }
    @Override no usages
    protected void applySelfEffects(Pokemon p) {
        // Снижение специальной атаки на 2 этапа
        p.setMod(Stat.SPECIAL_ATTACK, i: -2);
    }
}

```

Рисунок 11

```

package Moves;
import ru.ifmo.se.pokemon.*;

public class FocusEnergy extends StatusMove { no usages
    public FocusEnergy() { no usages
        super(Type.NORMAL, v: 0, v1: 0);
    }

    @Override
    protected String describe() {
        return "Применил Focus Energy";
    }
    @Override no usages
    protected void applySelfEffects(Pokemon p) {
        p.setMod(Stat.SPECIAL_ATTACK, i: +1);
    }
}

```

Рисунок 12

```

package Moves;
import ru.ifmo.se.pokemon.*;

public class FireBlast extends SpecialMove{ 1 usage
    public FireBlast(){ 1 usage
        super(Type.FIRE, v: 110, v1: 85);
    }

    @Override
    protected String describe(){
        return "Применил Fire Blast ";
    }

    @Override no usages
    protected void applyOppEffects(Pokemon p){
        // Проверяем вероятность поджигания
        if (Math.random() < 0.1)
            // Применяем эффект поджигания
            Effect.burn(p);
    }
}

```

Рисунок 13

```

package Moves;
import ru.ifmo.se.pokemon.*;

public class Facade extends PhysicalMove { 1 usage
    public Facade() { 1 usage
        super(Type.NORMAL, v: 70, v1: 100);
    }

    @Override
    protected String describe() {
        return "Применил Facade";
    }

    @Override no usages
    protected void applyOppDamage(Pokemon def, double damage) {
        // Получаем статус противника
        Status cond = def.getCondition();
        if (cond.equals(Status.BURN) || cond.equals(Status.POISON) || cond.equals(Status.PARALYZE)) {
            // Применяем урон x2
            def.setMod(Stat.HP, (int) Math.round(damage) * 2);
        }
        else {
            // Применяем обычный урон
            def.setMod(Stat.HP, (int) Math.round(damage));
        }
    }
}

```

Рисунок 14

```

package Moves;
import ru.ifmo.se.pokemon.*;

public class DreamEater extends SpecialMove { 1 usage
    public DreamEater() { 1 usage
        super(Type.PSYCHIC, v: 100, v1: 100);
    }

    @Override
    protected String describe() {
        return "Применил Dream Eater";
    }

    @Override no usages
    protected void applyOppDamage(Pokemon def, double damage) {
        // Узнаем статус противника
        Status opp = def.getCondition();
        if (opp.equals(Status.SLEEP)) {
            // Изменяем HP противника
            def.setMod(Stat.HP, (int) Math.round(damage));
        }
    }

    @Override no usages
    protected void applySelfEffects(Pokemon p) {
        // Вычисляем количество здоровья покемона
        int vosHP = (int) (p.getStat(Stat.HP) - p.getHP() / 2);
        // Восстанавливаем здоровье
        p.setMod(Stat.HP, vosHP);
    }
}

```

Рисунок 15

```

package Moves;
import ru.ifmo.se.pokemon.*;

public class DoubleTeam extends StatusMove { 1 usage
    public DoubleTeam() { 1 usage
        super(Type.NORMAL, v: 0, v1: 0);
    }

    @Override
    protected String describe() {
        return "Применил Double Team";
    }

    @Override no usages
    protected void applySelfEffects(Pokemon p) {
        // Проверяем текущее значение уклончивости
        if (p.getStat(Stat.EVASION) < 6) { // Максимальное значение уклончивости - 6
            p.setMod(Stat.EVASION, +1);
        }
    }
}

```

Рисунок 16

```

package Moves;
import ru.ifmo.se.pokemon.*;

public class DazzlingGleam extends SpecialMove{ 1 usage
    public DazzlingGleam(){ 1 usage
        super(Type.FAIRY, v: 80, v1: 100);
    }
    @Override
    protected String describe(){
        return "Применил Dazzling Gleam";
    }
    @Override no usages
    protected void applySelfDamage(Pokemon opp, double damage) {
        opp.setMod(Stat.HP, (int) damage);
    }
}

```

Рисунок 17

```

package Moves;
import ru.ifmo.se.pokemon.*;

public class AncientPower extends SpecialMove{ 1 usage
    public AncientPower(){ 1 usage
        super(Type.ROCK, v: 60, v1: 100);
    }
    @Override
    protected String describe(){
        return "Применил Ancient Power";
    }
    @Override no usages
    protected void applySelfEffects(Pokemon p){
        // Проверяем шанс
        if (Math.random() < 0.1)
            // Повышаем всю статистику пользователя
            p.setMod(Stat.ATTACK, i: 1);
            p.setMod(Stat.DEFENSE, i: 1);
            p.setMod(Stat.SPECIAL_ATTACK, i: 1);
            p.setMod(Stat.SPECIAL_DEFENSE, i: 1);
            p.setMod(Stat.SPEED, i: 1);
    }
}

```

Рисунок 18

- Сборка на гелиус  
mkdir lab2 (создаем каталог на гелиусе)

cd lab2(заходим у меня на ноутбуке в каталог, где лежат мои классы)

scp -r src g:/home/studs/s466310/lab2 (рекурсивно копируем каталог в каталог на гелиусе)

```
scp -r lib g:/home/studs/s466310/lab2.1
```

```
cd lab2.1
```

```
javac -cp lib/Pokemon.jar -d classes src/Pokemon/*.java src/Move/*.java src/Main.java  
(компилируем) (-cp (или -classpath) указывает путь к библиотекам, которые  
необходимы для компиляции., -d указывает директори)
```

```
echo -e "Manifest-Version: 1.0\nMain-Class: Main \nClass-Path: lib/Pokemon.jar" >  
MANIFEST.mf (создаем MANIFEST.mf файл, сообщающий главный класс и  
библиотеки, которые использует приложение)
```

```
jar -cvfm app.jar MANIFEST.mf -C classes . (собираем .jar архив из этих .class файлов  
и MANIFEST.mf манифеста) (-c: указывает на создание нового JAR-файла, -v:  
включает подробный вывод, -f указывает, что следующий аргумент будет именем  
создаваемого файла, -C classes .: указывает, что нужно перейти в директорию classes  
и добавить все файлы из этой директории в JAR-файл (.) текущая директория, то есть  
все внутри classes)
```

```
java -jar app.jar (запускаем)
```

### **Результат работы программы**

Bouffalant из команды белых вступает в бой!  
Togepi из команды фиолетовых вступает в бой!  
Bouffalant Применил Swagger.  
Togepi Применил Facade.  
Bouffalant теряет 4 здоровья.  
Bouffalant промахивается  
Togepi Применил Dazzling Gleam.  
Bouffalant теряет 10 здоровья.  
Bouffalant теряет сознание.  
Ninetales из команды белых вступает в бой!  
Ninetales Применил Swagger.  
Togepi Применил Dazzling Gleam.  
Ninetales теряет 4 здоровья.  
Ninetales Применил Fire Blast .  
Togepi теряет 8 здоровья.  
Togepi растерянно попадает по себе.  
Togepi теряет 3 здоровья.  
Ninetales Применил Fire Blast .  
Togepi теряет 8 здоровья.  
Togepi теряет сознание.  
Togetic из команды фиолетовых вступает в бой!  
Ninetales Применил Fire Blast .  
Togetic теряет 8 здоровья.  
Togetic Применил Ancient Power.  
Ninetales теряет 9 здоровья.  
Togetic увеличивает защиту.  
Togetic увеличивает специальную атаку.  
Togetic увеличивает специальную защиту.  
Togetic увеличивает скорость.

Ninetales теряет сознание.  
Togekiss из команды белых вступает в бой!  
Togekiss Применил Facade.  
Togetic теряет 4 здоровья.  
Togetic Применил Facade.  
Togekiss теряет 3 здоровья.  
Togekiss Применил Dazzling Gleam.  
Togetic теряет 10 здоровья.  
Togetic теряет сознание.  
Vulpix из команды фиолетовых вступает в бой!  
Togekiss Применил Ancient Power.  
15Vulpix теряет 6 здоровья.  
Togekiss увеличивает защиту.  
Togekiss увеличивает специальную атаку.  
Togekiss увеличивает специальную защиту.  
Togekiss увеличивает скорость.  
Vulpix Применил Overheat.  
Togekiss теряет 8 здоровья.  
Togekiss Применил Ancient Power.  
Vulpix теряет 13 здоровья.  
Togekiss увеличивает атаку.  
Togekiss увеличивает защиту.  
Togekiss увеличивает специальную атаку.  
Togekiss увеличивает специальную защиту.  
Togekiss увеличивает скорость.  
Vulpix теряет сознание.  
В команде фиолетовых не осталось покемонов.  
Команда белых побеждает в этом бою!

## **Вывод**

Выполнение лабораторной работы по программированию позволило мне более углубленно познакомиться с языком программирования Java. Данная работа дала мне возможность применить полученные теоретические знания на практике, развить навыки работы с информацией. В процессе выполнения я научилась создавать пакеты, работать с объектами, классами, конструкторами и переопределять методы. Приобретённый опыт поможет мне при дальнейшем изучении языка Java.